



Número Racional

Para este ejercicio pueden asumir que $\div$ es división entera.

TAD Fracción {

obs numerador : $\mathbb{Z}$

obs denominador : $\mathbb{Z}$

proc nuevaFracción(in $n, d : \mathbb{Z}$) : Fracción {

requiere { $d \neq 0$ }

asegura { $res.numerador = n \wedge res.denominador = d$ }

}

proc igualdad(in $f1, f2 : Fracción$) : bool {

asegura { $res = true \leftrightarrow F_1.numerador \div F_1.denominador = F_2.numerador \div F_2.denominador$ }

}

proc suma(in $f1, f2 : Fracción$) : Fracción {

requiere { Verdadero }

asegura {

$res.numerador = f1.numerador \times f2.denominador + f2.numerador \times f1.denominador \wedge$

$res.denominador = f1.denominador \times f2.denominador$

}

}

proc resta(in $f1, f2 : Fracción$) : Fracción {

requiere { Verdadero }

asegura {

$res.numerador = f1.numerador \times f2.denominador - f2.numerador \times f1.denominador \wedge$

$res.denominador = f1.denominador \times f2.denominador$

}

}

proc multiplicacion(in $f1, f2 : Fracción$) : Fracción {

requiere { Verdadero }

asegura {

$res.numerador = f1.numerador \times f2.numerador \wedge$

$res.denominador = f1.denominador \times f2.denominador$

}

}

proc division(in $f1, f2 : Fracción$) : Fracción {

requiere { $f2.numerador \neq 0$ }

asegura {

$res.numerador = f1.numerador \times f2.denominador \wedge$

$res.denominador = f1.denominador \times f2.numerador$

}

}

}

Doble cola

```

TAD DobleCola⟨T⟩ {
  obs elems : seq⟨T⟩
  proc nuevaDobleCola() : DobleCola⟨T⟩ {
    requiere { Verdadero }
    asegura { |res.elems| = 0 }
  }

  proc igualdad(in c1, c2 : DobleCola⟨T⟩) : bool {
    asegura { res ↔ c1.elems = c2.elems }
  }

  proc estaVacía(in c : DobleCola⟨T⟩) : bool {
    requiere { Verdadero }
    asegura { res = True ↔ |c.elems| = 0 }
  }

  proc encolarAtrás(inout c : DobleCola⟨T⟩, in elem : T) {
    requiere { c = C0 }
    asegura { c.elems = C0.elems ++ ⟨e⟩ }
  }

  proc encolarAdelante(inout c : DobleCola⟨T⟩, in elem : T) {
    requiere { p = P0 }
    asegura { agregadoAdelante(C0, c, e) }
  }

  – Tomo ÷ como división entera
  proc desencolar(inout c : DobleCola⟨T⟩) : T {
    requiere { c = C0 ∧ |c.elems| > 0 }
    asegura { res = C0.elems[|C0.elems| ÷ 2] ∧ sacoElDelMedio(C0, c) }
  }

  pred agregadoAdelante(C0, c : DobleCola⟨T⟩, e : T) {
    c.elems[0] = e ∧ |c.elems| = |C0.elems| + 1 ∧L
    (∀i : ℤ) (0 ≤ i < |c.elems| →L c.elems[i] = C0.elems[i - 1])
  }

  pred sacoElDelmedio(C0, c : DobleCola⟨T⟩) {
    |c.elems| = |C0.elems| - 1 ∧L (
      (∀i : ℤ) (0 ≤ i < |c.elems| ÷ 2 →L c.elems[i] = C0.elems[i]) ∧
      (∀i : ℤ) (|c.elems| ÷ 2 ≤ i < |c.elems| →L c.elems[i] = C0.elems[i + 1])
    )
  }
}

```

Posición ES tupla $\langle \mathbb{Z}, \mathbb{Z} \rangle$
 Matriz $\langle T \rangle$ ES seq $\langle \text{seq} \langle T \rangle \rangle$

```

TAD Buscaminas {
  obs bombas : Matriz<bool>
  obs descubiertas : Matriz<bool>
  obs banderines : Matriz<bool>

  proc igualdad(in  $B_1 : \text{Buscaminas}$ , in  $B_2 : \text{Buscaminas}$ ) : bool {
    asegura {  $res = \text{true} \leftrightarrow$ 
       $B_1.bombas = B_2.bombas \wedge$ 
       $B_1.descubiertas = B_2.descubiertas \wedge$ 
       $B_1.banderines = B_2.banderines$ 
    }
  }

  proc nuevoJuego(in  $t : \text{Matriz} \langle \text{bool} \rangle$ ) : Buscaminas {
    requiere {  $\text{esMatriz}(t) \wedge |t| > 0 \wedge_L |t[0]| > 0$  }
    asegura {
       $res.bombas = t \wedge \text{mismasDimensiones}(res.descubiertas, t) \wedge$ 
       $\text{todoFalse}(res.descubiertas) \wedge res.banderines = res.descubiertas$ 
    }
  }

  pred esMatriz( $s : \text{Matriz} \langle \text{bool} \rangle$ ) {
     $(\exists l : \mathbb{Z}) ((\forall i : \mathbb{Z}) (0 \leq i < |s| \rightarrow_L |s[i]| = l))$ 
  }

  pred mismasDimensiones( $A : \text{Matriz} \langle \text{bool} \rangle, B : \text{Matriz} \langle \text{bool} \rangle$ ) {
     $|A| = |B| \wedge_L (\forall i : \mathbb{Z}) (0 \leq i < |A| \rightarrow_L |A[i]| = |B[i]|)$ 
  }

  pred todoFalse( $A : \text{Matriz} \langle \text{bool} \rangle$ ) {
     $(\forall i, j : \mathbb{Z}) (\text{enRango}(i, j, A) \rightarrow_L A[i][j] = \text{false})$ 
  }

  pred enRango( $i, j : \mathbb{Z}, A : \text{Matriz} \langle \text{bool} \rangle$ ) {
     $0 \leq i < |A| \wedge_L 0 \leq j < |A[0]|$ 
  }

  proc estadoDelJuego(in  $b : \text{Buscaminas}$ ) : string {
    requiere { Verdadero }
    asegura {
       $(\text{reventó}(b) \rightarrow res = \text{"carita muerta"}) \wedge$ 
       $((\neg \text{reventó}(b) \wedge \text{ganó}(b)) \rightarrow res = \text{"carita cool"}) \wedge$ 
       $((\neg \text{reventó}(b) \wedge \neg \text{ganó}(b)) \rightarrow res = \text{"carita feliz"})$ 
    }
  }

  pred reventó( $b : \text{Buscaminas}$ ) {
     $(\exists i, j : \mathbb{Z}) (\text{enRango}(i, j, b.bombas) \wedge_L (b.bombas[i][j] = \text{true} \wedge b.descubiertas[i][j] = \text{true}))$ 
  }

  pred ganó( $b : \text{Buscaminas}$ ) {
     $(\forall i, j : \mathbb{Z}) (\text{enRango}(i, j, b.bombas) \rightarrow_L (b.bombas[i][j] = \text{false} \rightarrow b.descubiertas[i][j] = \text{true}))$ 
  }

  proc jugar(inout  $b : \text{Buscaminas}$ , in  $i, j : \mathbb{Z}$ ) {
    requiere {
       $b = B0 \wedge$ 
       $\neg(\text{reventó}(b) \vee \text{jugóTodosLosVacíos}(b)) \wedge$ 
       $\text{enRango}(i, j, b.bombas) \wedge_L b.descubiertas[i][j] = \text{false} \wedge b.banderines[i][j] = \text{false}$ 
    }
    asegura {

```

```

    b.bombas = B0.bombas  $\wedge$ 
    b.banderines = B0.banderines  $\wedge$ 
    mismasDimensiones(b.descubiertas, B0.descubiertas)  $\wedge_L$  (
        b.descubiertas[i][j] = true  $\wedge$ 
        descubiertasSeMantienen(B0, b)  $\wedge$ 
        descubiertasEnCadena(B0, b, i, j)
    )
}
}

pred descubiertasSeMantienen(B0, b : Buscaminas) {
    ( $\forall k, l : \mathbb{Z}$ ) (enRango(k, l, b.bombas)  $\rightarrow_L$  (B0.descubiertas[k][l] = true  $\rightarrow$  b.descubiertas[k][l] = true))
}

pred descubiertasEnCadena(B0, b : Buscaminas, i, j :  $\mathbb{Z}$ ) {
    ( $\forall k, l : \mathbb{Z}$ ) (
        enRango(k, l, b.bombas)  $\rightarrow_L$  (B0.descubiertas[k][l] = false  $\rightarrow$ 
        hayCaminoLibreEntre(b, i, j, k, l)  $\leftrightarrow$  b.descubiertas[k][l] = true)
    )
}

pred hayCaminoLibreEntre(b : Buscaminas, i_d, j_d, i_h, j_h :  $\mathbb{Z}$ ) {
    b.bombas[i_d][j_d] = false  $\wedge$  b.bombas[i_h][j_h] = false  $\wedge$ 
    ( $\exists s : \text{seq}(\text{Posición})$ ) (
        esCaminoEn(s, b)  $\wedge$  |s| > 0  $\wedge_L$  s[0] =  $\langle i_d, j_d \rangle \wedge$  s[|s| - 1] =  $\langle i_h, j_h \rangle \wedge$ 
        ( $\forall k : \mathbb{Z}$ ) (0  $\leq k < |s| - 1 \rightarrow_L$  bombasAlrededor(s[k]_1, s[k]_2, b) = 0)
    )
}

pred esCaminoEn(s : seq(Posición), b : Buscaminas) {
    ( $\forall i : \mathbb{Z}$ ) (
        1  $\leq i < |s| \rightarrow_L$  (enRango(s[i]_1, s[i]_2, b.bombas)  $\wedge$  sonAdyacentes(s[i - 1], s[i]))
    )
}

pred sonAdyacentes(p, q : Posición) {
    p_1 - 1  $\leq q_1 \leq p_1 + 1 \wedge$  p_2 - 1  $\leq q_2 \leq p_2 + 1 \wedge \neg(p_1 = q_1 \wedge p_2 = q_2)$ 
}

aux bombasAlrededor(i, j :  $\mathbb{Z}$ , b : Buscaminas) :  $\mathbb{Z}$  =
     $\sum_{k=i-1}^{i+1} \left( \sum_{l=j-1}^{j+1} \text{IfThenElse}(\text{enRango}(k, l, b.bombas) \wedge_L b.bombas[k][l] = \text{true}, 1, 0) \right)$ 

proc bombasAlrededor(in b : Buscaminas, in i, j :  $\mathbb{Z}$ ) :  $\mathbb{Z}$  {
    requiere { enRango(i, j, b.bombas)  $\wedge_L$  b.descubiertas[i][j] = true  $\wedge$  b.bombas[i][j] = false }
    asegura { res = bombasAlrededor(i, j, b) }
}

proc flipBanderín(inout b : Buscaminas, in i, j :  $\mathbb{Z}$ ) {
    requiere { b = B0  $\wedge$  enRango(i, j, b.banderines)  $\wedge_L$  b.descubiertas[i][j] = false }
    asegura {
        b.bombas = B0.bombas  $\wedge$ 
        b.descubiertas = B0.descubiertas  $\wedge$ 
        mismasDimensiones(b.banderines, B0.banderines)  $\wedge_L$  (
            b.banderines[i][j] =  $\neg$ B0.banderines[i][j]  $\wedge$  otrasBanderinesSeMantienen(B0, b, i, j)
        )
    }
}

pred otrasBanderinesSeMantienen(B0, b : Buscaminas, i, j :  $\mathbb{Z}$ ) {
    ( $\forall k, l : \mathbb{Z}$ ) ((enRango(k, l, b.banderines)  $\wedge$  k  $\neq i \vee l \neq j$ )  $\rightarrow_L$  b.banderines[i][j] = B0.banderines[i][j])
}
}

```